

```
En [82]: importar numpy como np
importar pandas como pd
importar matplotlib.pyplot como plt
importar seaborn como sns
```

```
En [83]: ! Príncipe
del espectáculo Pip!
```

Nombre: príncipe

Versión: 0.16.1

Resumen: Análisis factorial en Python: PCA, CA, MCA, MFA, FAMD, GPA

Página principal:

Autor:

Correo electrónico del autor: Max Halford <maxhalford25@gmail.com>

Licencia:

Ubicación: C:\Usuarios\Patricia\anaconda3\Lib\site-packages

Requiere: altair, pandas, scikit-learn

Requerido por:

```
En [84]: príncipe de importación
```

```
En [85]: datos1 = pd.read_csv ( " datos2_pfb2.csv " )
```

```
En [86]: datos1 . cabeza ( 2 )
```

Salida[...

	distrito	estado_meteorológico	sexo	alcohol positivo	droga positiva	franja_horaria	clasificaci
0	RETIRO	Despejado	Hombre	norte	norte	Noche	
1	RETIRO	Despejado	Hombre	norte	norte	Noche	

```
En [87]: datos2 = datos1 . copiar ()
```

```
En [88]: datos2 = datos2 . drop ( columnas = [ 'estado_meteorológico' , 'grupos_por_ed
```

```
En [89]: datos2 . cabeza ( 2 )
```

Salida[...

	sexo	alcohol positivo	droga positiva	franja_horaria	clasificación_accidente	clasificación_vehículo
0	Hombre	norte	norte	Noche	Colisión	Turismo/Furgone
1	Hombre	norte	norte	Noche	Colisión	Turismo/Furgone

```
En [90]: data2 [ 'clasificacion_lesividad' ] . valor_cuenta ()
```

Salida[...

```
clasificacion_lesividad
Se desconoce 21990
Sin asistencia sanitaria 16111
Con asistencia sanitaria 11194
Fallecido 24 horas 29
Nombre: count, tipo de dato: int64
```

```
En [91]: data2 [ 'clasificacion_lesividad' ] = data2 [ 'clasificacion_lesividad' ] . re
        'Fallecido 24 horas' , 'Con asistencia sanitaria'
        )
```

```
En [92]: data2 [ 'clasificacion_lesividad' ] . valor_cuenta ()
```

Salida[...

```
clasificacion_lesividad
Se desconoce 21990
Sin asistencia sanitaria 16111
Con asistencia sanitaria 11223
Nombre: count, tipo de dato: int64
```

```
En [93]: # proporciones = data2[data2['clasificacion_lesividad'] != 'Se desconoce'] \
        # ['clasificacion_lesividad'].value_counts(normalize=True)

        # máscara = data2['clasificacion_lesividad'] == 'Se desconoce'

        # data2.loc[mask, 'clasificacion_lesividad'] = np.random.choice(
        #     ['Sin asistencia sanitaria', 'Con asistencia sanitaria'],
        #     size=mask.sum(),
        #     p=[proporciones['Sin asistencia sanitaria'], proporciones['Con asistencia
        # )
```

```
In [94]: mask = data2['clasificacion_lesividad'] != 'Se desconoce'
        data2 = data2.loc[mask,:]
```

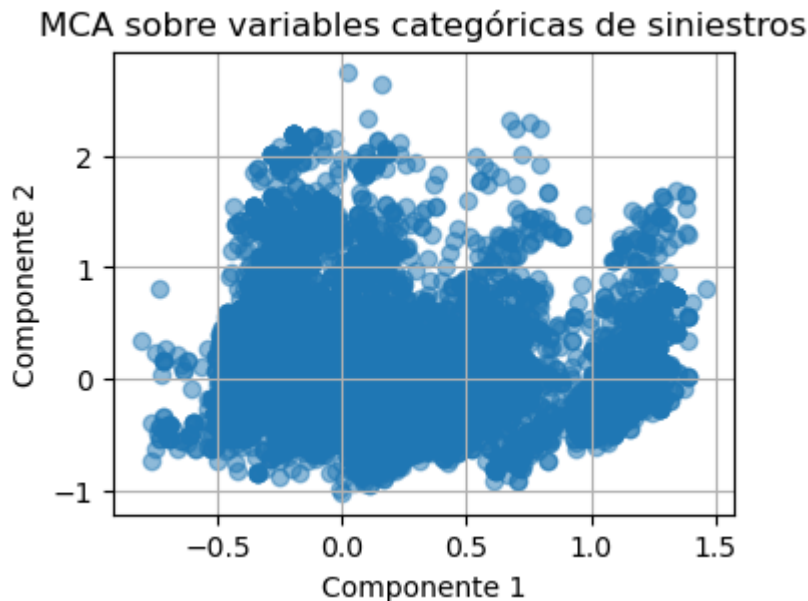
MCA

```
In [95]: columnas_mca = ['mapa_zonas', 'franja_horaria', 'clasificacion_accidente', 'edades
        data_mca = data2[columnas_mca].dropna()
```

```
In [96]: mca = prince.MCA(n_components=2, random_state=42)
        mca = mca.fit(data_mca)
```

```
In [97]: mca_resultados = mca.transform(data_mca)
```

```
In [98]: plt.figure(figsize=(4,3))
plt.scatter(mca_resultados[0], mca_resultados[1], alpha=0.5)
plt.xlabel('Componente 1')
plt.ylabel('Componente 2')
plt.title('MCA sobre variables categóricas de siniestros')
plt.grid(True)
plt.show()
```



```
In [99]: mca = prince.MCA(n_components=len(data_mca.columns), random_state=42)
mca = mca.fit(data_mca)
```

```
In [100... eigenvalues = mca.eigenvalues_
```

```
In [101... inercia = [val / sum(eigenvalues) for val in eigenvalues]
```

```
In [102... inercia_acumulada = pd.Series(inercia).cumsum()
inercia_acumulada
```

```
Out[102... 0    0.147132
1    0.265322
2    0.375639
3    0.472067
4    0.566637
5    0.657837
6    0.746194
7    0.832076
8    0.916856
9    1.000000
dtype: float64
```

```
In [103... #Los primeros 7 componentes explican más del 76 % de la variabilidad (inercia ac
```

```
In [104... print(data2.columns[8])
print(data2.columns[9])
```

```
clima
edades
```

```
In [105... data2 = data2.drop(data2.columns[[8, 9]], axis=1)
```

```
In [106... data2.head(2)
```

```
Out[106... 
```

	sexo	positiva_alcohol	positiva_droga	franja_horaria	clasificacion_accidente	clasi
0	Hombre	N	N	Noche	Colisión	T
1	Hombre	N	N	Noche	Colisión	T

◀ ▶

1. Clasificación binaria: ¿Hubo asistencia médica? Random Forest

```
In [107... from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
```

```
In [108... dummies = pd.get_dummies(data2, drop_first=True)
```

```
In [109... print(dummies.columns.tolist())
```

```
['sexo_Hombre', 'sexo_Mujer', 'positiva_alcohol_S', 'positiva_droga_S', 'franja_h
oraria_Noche', 'franja_horaria_Tarde', 'clasificacion_accidente_Caída', 'clasific
acion_accidente_Choque/Obstáculo/Salida via', 'clasificacion_accidente_Colisión',
'clasificacion_accidente_Otro', 'clasificacion_vehiculo_Moto/Ciclomotor', 'clasif
icacion_vehiculo_Otro', 'clasificacion_vehiculo_Pesado/Especial', 'clasificacion_
vehiculo_Sin motor', 'clasificacion_vehiculo_Transporte colectivo', 'clasificacio
n_vehiculo_Turismo/Furgoneta', 'clasificacion_lesividad_Sin asistencia sanitari
a', 'mapa_zonas_este', 'mapa_zonas_norte', 'mapa_zonas_oeste', 'mapa_zonas_sur']
```

```
In [110... X = dummies.drop(columns=['clasificacion_lesividad_Sin asistencia sanitaria'])
y = dummies['clasificacion_lesividad_Sin asistencia sanitaria']
```

```
In [111... X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
```

```
In [112... modelo = RandomForestClassifier(n_estimators=100, random_state=42, max_depth= 5)
modelo.fit(X_train, y_train)
```

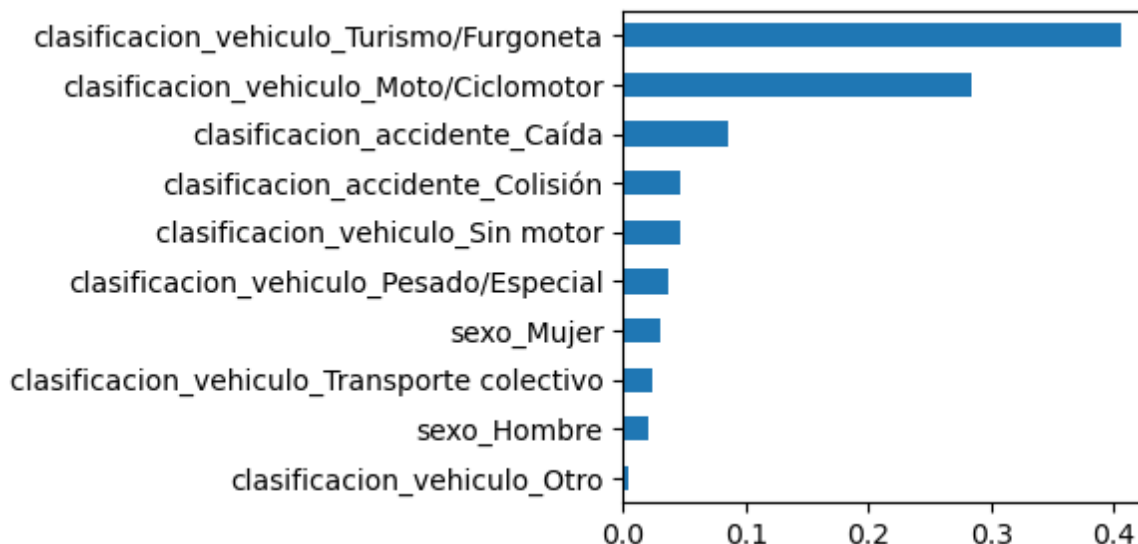
```
Out[112... 
```

▼ **RandomForestClassifier** ⓘ ?

RandomForestClassifier(max_depth=5, random_state=42)

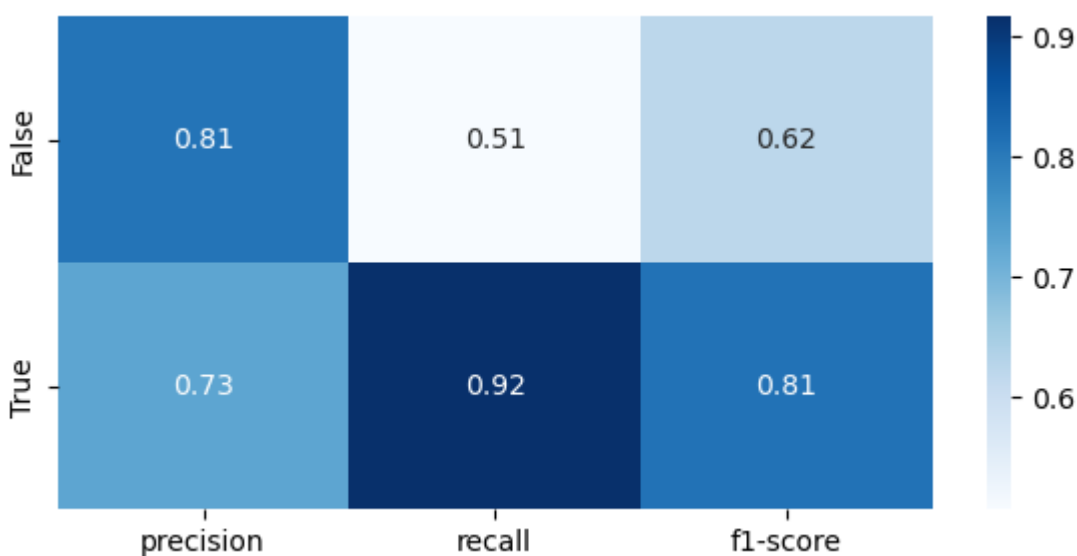
```
In [113... y_pred = modelo.predict(X_test)
```

```
In [114... importances = pd.Series(modelo.feature_importances_, index=X.columns)
plt.figure(figsize=(6,3))
importances.sort_values().tail(10).plot(kind='barh')
plt.tight_layout()
plt.show()
```



```
In [115... resumen = classification_report(y_test, y_pred, output_dict=True)
df = pd.DataFrame(resumen).transpose().iloc[:2, :3]

plt.figure(figsize=(6, 3))
sns.heatmap(df, annot=True, cmap='Blues', fmt=".2f")
plt.tight_layout()
plt.show()
```



```
In [116... from sklearn.metrics import accuracy_score

accuracy_score(y_test, y_pred)
```

Out[116... 0.7495427386904036

```
In [ ]: #El modelo identifica correctamente la mayoría de los casos con asistencia sanit
# pero tiene más dificultad para detectar los que no la tuvieron
#posiblemente influido por la integración de registros originalmente clasificado
#cuya redistribución se realizó según criterios proporcionales al no disponer de
#El modelo acierta en casi el 75 % de los casos,
# Lo que indica un buen nivel de precisión en sus predicciones.
```

Calasificación multiclase: ¿Qué tipo de accidente fue?

Decision Tree

```
In [118... from sklearn.tree import DecisionTreeClassifier
```

```
In [119... X_acc = dummies.drop(columns=['clasificacion_accidente_Colisión'])
y_acc = dummies['clasificacion_accidente_Colisión']
```

```
In [120... X_acc_train, X_acc_test, y_acc_train, y_acc_test = train_test_split(X_acc, y_acc)
modelo_multi = DecisionTreeClassifier()
modelo_multi.fit(X_acc_train, y_acc_train)
```

```
Out[120... DecisionTreeClassifier
DecisionTreeClassifier()
```

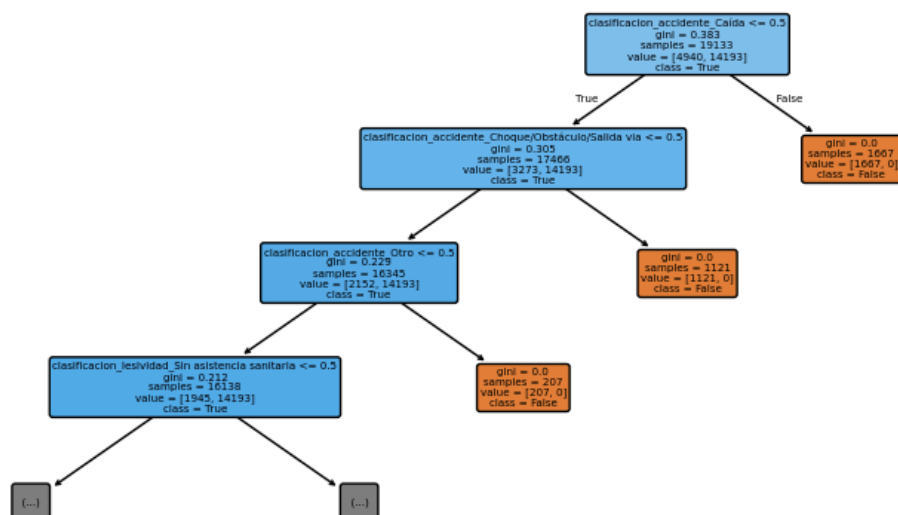
```
In [121... y_acc_pred = modelo_multi.predict(X_acc_test)
```

```
In [122... print(classification_report(y_acc_test, modelo_multi.predict(X_acc_test)))
```

	precision	recall	f1-score	support
False	0.94	0.61	0.74	2106
True	0.88	0.99	0.93	6095
accuracy			0.89	8201
macro avg	0.91	0.80	0.84	8201
weighted avg	0.90	0.89	0.88	8201

```
In [123... from sklearn.tree import plot_tree

plt.figure(figsize=(10, 5))
plot_tree(modelo_multi,
          feature_names=X_acc.columns,
          class_names=["False", "True"],
          filled=True,
          rounded=True,
          max_depth=3
        )
plt.show()
```



```
In [124... accuracy_score(y_acc_test, y_acc_pred)
```

```
Out[124... 0.890745031093769
```

```
In [ ]: #El modelo presenta un rendimiento muy bueno en la clasificación del tipo de acc  
#casi siempre acierta sobre el tipo de accidente, en el 89% de los casos de coli
```

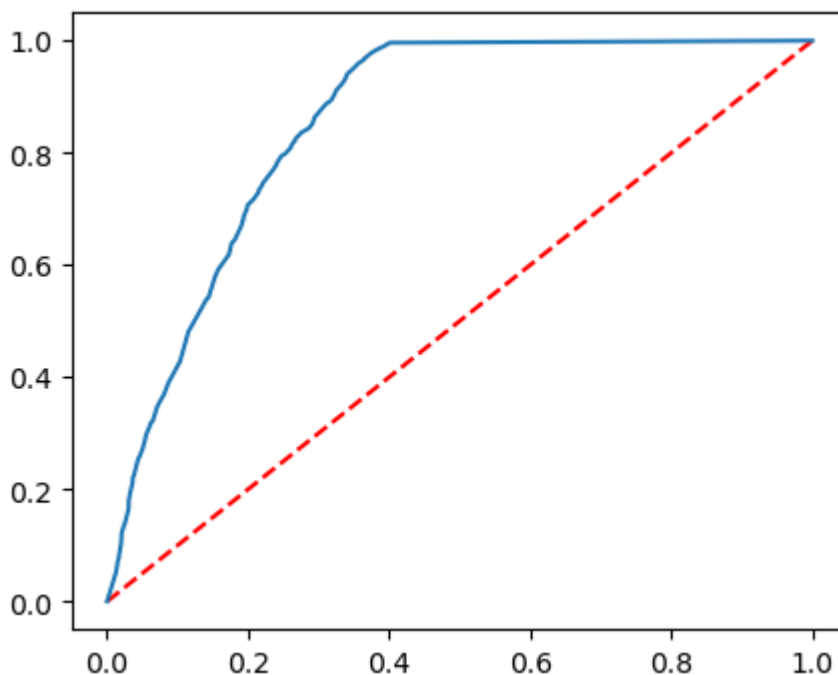
Curva de ROC

```
In [126... y_proba = modelo_multi.predict_proba(X_acc_test)[:, 1]
```

```
In [127... from sklearn.metrics import roc_curve, roc_auc_score  
  
especificidad_inv, sensibilidad, threshold = roc_curve(y_acc_test, y_proba)
```

```
In [128... recta_x = [i * 0.01 for i in range(101)]  
recta_y = recta_x  
plt.figure(figsize=(5, 4))  
sns.lineplot(x = recta_x, y = recta_y, ls = '--', color = 'red')  
sns.lineplot(x = especificidad_inv, y = sensibilidad)
```

```
Out[128... <Axes: >
```



```
In [129... roc_auc_score(y_test, y_proba)
```

```
Out[129... 0.6788867435849235
```

```
In [ ]: #El modelo de árbol de decisión aplicado al tipo de accidente alcanza un AUC de  
# lo que indica una capacidad moderada para distinguir entre categorías.  
# está captando patrones útiles, aunque no de forma óptima.
```